CodeProctor

A Coding Assessment and Malpractice Detection Platform

Team No: 15

Team Members:

Abhinand M A (04)

Mathew Punnen Chandy (40)

Sreelakshmi K S (56)

Abstract

CodeProctor is **a web-based coding assessment and malpractice detection platform** that integrates competitive programming with transparent and explainable online examination proctoring. The platform supports two user roles: **Admin** and **User**. Administrators publish public practice problems for skill development, while any user can organize timed coding contests with configurable proctoring modes, including Normal mode, Full-Screen Enforcement, and Computer Vision (CV)-based webcam monitoring. Users can solve practice problems, create their own coding contests, or participate in contests created by other users, while complying with the selected proctoring requirements.

The primary objective of CodeProctor is to provide a secure and fair online coding environment by combining automated code evaluation with explainable malpractice detection. Instead of generating a simple cheating flag, the platform records a timestamped timeline of suspicious activities such as browser tab switches, full -screen exits, face absence, multiple-face detection, and prolonged gaze diversion. These events are captured using browser APIs and analyzed through a Python-based FastAPI microservice integrated with OpenCV. The user who created the contest can review the complete event history and make transparent, evidence-based decisions regarding participant misconduct.

To ensure secure execution of user-submitted programs, source code is executed inside isolated and resource-limited **Docker** containers. Submission requests are processed asynchronously through a **Redis-backed job queue**, preventing malicious or faulty programs from affecting the host system while supporting multiple programming languages. The platform is developed using **Next.js** as a unified full-stack framework, handling both the frontend and backend within a single codebase, **PostgreSQL** for persistent data storage, and **Redis** for caching and asynchronous task management.

Overall, CodeProctor provides a comprehensive solution for secure online coding assessments by integrating automated code evaluation, configurable proctoring, sandboxed code execution, and explainable malpractice detection within a scalable, modular, and user-friendly platform.

Technologies Used

1. **Frontend**
   - Next.js
   - HTML5
   - CSS3
   - TypeScript
2. **Backend**
   - Node.js
   - Next.js API Routes
   - REST APIs
3. **Database**
   - PostgreSQL
   - Prisma
4. **Caching and Job Queue**
   - Redis
5. **Code Execution**
   - Docker Containers
   - Secure Sandbox Environment
   - Multi-language Code Execution
6. **Computer Vision and Proctoring**
   - Python
   - FastAPI
   - OpenCV
   - Browser APIs
7. **Development Tools**
   - Git
   - GitHub
   - Postman
   - Visual Studio Code